EVOLVE

ACADEMY

Guide technique

Architecture applicative et gestion Evolve Staff

Lisez ce guide avant de modifier le code ou de soumettre une Pull Request.

EV-2026-STAFF

Groupe REV · Division Éducation et Technologie · Octobre 2026 · Version 1.0.0

AVANT DE COMMENCER

Section 0 sur 16 : cadrage et principes directeurs

1 Staff CMS Flutter	2 Supabase Partagé	3 Rôles Staff	4 Edge Functions
Dart 3.9 / Flutter 3.35 (Web & Windows)	Auth, Postgres, RLS et Storage unifiés	teacher et admin (étudiants rejetés)	Deno / TypeScript pour actions privilégiées

1. **Dashboard Staff (Flutter) :** Application multiplateforme (SPA Web Vercel et binaire Windows) pilotant cours, devoirs et finances.
2. **Base de données partagée :** Supabase PostgreSQL partagé en direct avec le portail étudiant Next.js (`evolve-web`).
3. **Ségrégation stricte des rôles :** Seuls les profils avec le rôle `teacher` ou `admin` sont autorisés à franchir l'écran de login.
4. **Zéro clé secrète dans Flutter :** Jamais de clé `service_role` dans le code Flutter ; les actions globales passent par Edge Functions.

À SAVOIR ABSOLUMENT

Le schéma SQL et les migrations sont centralisés : tout ajout de colonne ou trigger doit être versionné sous `supabase/migrations/`.

Les secrets API et tokens de service ne figurent JAMAIS côté client : seule la clé anonyme publique est déclarée dans `lib/main.dart`.

Toute modification d'écran ou de service doit passer la suite d'assurance qualité locale via `scripts/check.sh`.

README.md **Lignes 28-43**

```
28: L’application est préconfigurée pour le projet Supabase partagé avec Evolve
29: School. Si vous utilisez un autre environnement, adaptez la configuration
30: publique Supabase dans `lib/main.dart`. **N’ajoutez jamais de clé `service_role`
31: ou de secret Chargily à l’application Flutter.**
32:
33: ## Se connecter
34:
35: 1. Connectez-vous avec l’adresse e-mail et le mot de passe du compte.
36: 2. Le compte doit exister dans Supabase Auth et son profil doit avoir le rôle
37:    `teacher` ou `admin`.
38: 3. Si l’accès est refusé, demandez à un administrateur de vérifier le rôle dans
39:    le profil du compte.
40:
41: Les champs de connexion acceptent la touche Entrée, vérifient le format de
42: l’adresse e-mail et permettent d’afficher temporairement le mot de passe.
```

SOMMAIRE

Table des matières générale du référentiel Staff

1 Partie I : Config	2 Partie II : Métier	3 Partie III : Admin	4 Partie IV : Ops
Architecture, runtime et modularité (p. 4-6)	Auth, cours, leçons et devoirs (p. 7-13)	Temps réel, users, finances DZD (p. 14-17)	RLS, tests, build et index (p. 18-25)

1. **Partie I : Architecture & Fondations (p. 4 à 6) :** Flux de données, analyse de pubspec.yaml, découpage lib/ en screens et services.
2. **Partie II : Modules Pédagogiques (p. 7 à 13) :** Login RBAC, dashboard adaptatif, cours, leçons, formations, devoirs et progression.
3. **Partie III : Administration & Trésorerie (p. 14 à 17) :** Messagerie Realtime, gestion des rôles, grand livre DZD et export CSV.
4. **Partie IV : Sécurité, Déploiement & Index (p. 18 à 25) :** Edge Functions, triggers RLS, scripts CI, build Vercel et index de code.

ORGANISATION DE LA LECTURE

Une page = un sujet : Chaque fiche synthétise structure, responsabilités et extrait de code réel numéroté.

Les fonctionnalités réservées aux administrateurs (modération, grand livre financier, gestion RBAC) sont clairement balisées.

L'annexe finale référence exhaustivement l'intégralité des fichiers de l'application Flutter et de son backend Supabase.

FILE_CONTENT_REPORT.txt **Lignes 12-32**

```
12: FLUTTER APPLICATION AND SCREENS
13: lib/main.dart — Initializes Flutter and Supabase and starts the Evolve Staff app at login.
14: lib/screens/assignments_screen.dart — Lists/filters course assignments, creates assignments, and supports submission review and grad
15: lib/screens/course_details_screen.dart — Shows course details and lessons, with links to editing, lesson management, and course file
16: lib/screens/course_files_screen.dart — Lists course file records and uploads files to Supabase Storage with saved metadata.
17: lib/screens/courses_screen.dart — Lists, searches, and filters courses; includes course editing and administrator moderation actions
18: lib/screens/create_course_screen.dart — Course creation form for description, pricing, category, and publication fields.
19: lib/screens/dashboard_screen.dart — Role-aware staff navigation/dashboard with statistics, activity, shortcuts, and sign-out.
20: lib/screens/edit_course_screen.dart — Edits course details, pricing, and publication fields and saves to Supabase.
21: lib/screens/formations_screen.dart — Lists formations/series, creates them, and shows associated courses.
22: lib/screens/lesson_details_screen.dart — Displays lesson title, order, description, and edit navigation.
23: lib/screens/lessons_screen.dart — Lists lessons with course context and provides course/search filters and lesson creation.
24: lib/screens/login_screen.dart — Validates credentials, signs in with Supabase Auth, checks staff role, and routes authorized users.
25: lib/screens/messages_screen.dart — Displays student conversations with search/unread filters and refresh on real-time message events
26: lib/screens/revenue_screen.dart — Displays recorded revenue, refunds, expenses, enrollment counts, and course totals.
```

ARCHITECTURE GLOBALE

Section 1 sur 16 : flux de données et responsabilités

1 Client Flutter	2 Services & DTOs	3 Postgres + RLS	4 Edge Functions
Material 3 / State StatefulWidgets	Couche d'abstraction et validation stricte	Sécurité en base filtrée par JWT	Microservices Deno à privilèges élevés

1. **Interface Client (lib/screens) :** Composants réactifs Flutter Material 3 gérant formulaires, listes et états de chargement.
2. **Couche Services (lib/services) :** Isole les appels Supabase, transforme les réponses JSON et lève des exceptions typées.
3. **PostgreSQL & Row-Level Security :** Toutes les requêtes directes sont soumises aux politiques RLS selon le rôle utilisateur.
4. **Edge Functions Supabase :** Exécutent les opérations d'agrégation d'analytics et les modifications sécurisées de profils.

PRINCIPES ARCHITECTURAUX

Séparation stricte UI / Données : un écran n'exécute jamais de requêtes complexes directement s'il existe un service dédié.

Chaque modèle de données valide ses champs à l'instanciation (interdiction des montants négatifs ou identifiants nuls).

L'application s'adapte dynamiquement selon le rôle transmis au login (`teacher` ou `admin`).

lib/main.dart — **Lignes 1-32**

```
1: import 'package:flutter/material.dart';
2: import 'package:supabase_flutter/supabase_flutter.dart';
3: import 'package:evolve_staff/screens/login_screen.dart';
4:
5: Future main() async {
6:   WidgetsFlutterBinding.ensureInitialized();
7:
8:   const supabaseUrl = String.fromEnvironment(
9:     'SUPABASE_URL', defaultValue: 'https://ficrjocgrcghxrdpwbbw.supabase.co',
10:   );
11:   const supabaseAnonKey = String.fromEnvironment(
12:     'SUPABASE_PUBLISHABLE_KEY', defaultValue: 'sb_publishable_4iPyk0EvUvT2OfKrDbhQxA_bvOhv2P2',
13:   );
14:   await Supabase.initialize(url: supabaseUrl, publishableKey: supabaseAnonKey);
12:
13:   runApp(const EvolveStaffApp());
14: }
15:
16: class EvolveStaffApp extends StatelessWidget {
17:   const EvolveStaffApp({super.key});
18:
19:   @override
20:   Widget build(BuildContext context) {
21:     return MaterialApp(
22:       debugShowCheckedModeBanner: false,
23:       title: 'Evolve Staff',
24:       theme: ThemeData(
25:         fontFamily: 'Inter',
26:         useMaterial3: true,
27:         colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFF84CC16)),
28:       ),
```

DÉPENDANCES ET RUNTIME

Section 2 sur 16 : analyse de pubspec.yaml et SDK

❶ **SDK Dart / Flutter** Dart ^3.9.2 Flutter 3.35.x	❷ **supabase_flutter** ^2.17.2 : Auth, DB, Storage et Realtime	❸ **file_picker** ^11.0.3 : Sélection de fichiers multiplateforme	❹ **package web** ^1.1.1 : W3C JS Interop moderne pour Flutter Web

❶ **Environnement Dart 3.9 :** Typage strict, sound null-safety intégrale et compilation AOT / WASM optimisée.
❷ **supabase_flutter (^2.17.2) :** Pilote la session JWT, la persistance locale chiffrée et les flux d'écoute WebSocket.
❸ **file_picker (^11.0.3) :** Abstraction d'accès aux fichiers locaux pour l'upload de ressources PDF, ZIP et vidéos.
❹ **web (^1.1.1) :** Remplace l'ancien `dart:html` déprécié par le nouveau standard Dart JS Interop pour le Web.

BONNES PRATIQUES ET VERROUILLAGE

Le fichier `pubspec.lock` est strictement versionné dans Git pour garantir la reproductibilité des builds en CI/CD.

L'installation des paquets en intégration continue utilise obligatoirement le flag `--enforce-lockfile`.

Les dépendances Dart HTML sont isolées par compilation conditionnelle pour éviter de briser la compilation Windows native.

pubspec.yaml **Lignes 21-51**

```
21: environment:
22:   sdk: ^3.9.2
23:
24: dependencies:
25:   flutter:
26:     sdk: flutter
27:
28:   # Icônes vectorielles standard
29:   cupertino_icons: ^1.0.8
30:
31:   # Client officiel Supabase (Auth, PostgREST, Storage, Realtime)
32:   supabase_flutter: ^2.17.2
33:
34:   # Sélecteur de fichiers de cours et devoirs
35:   file_picker: ^11.0.3
36:
37:   # Standard moderne W3C Web Interop pour téléchargements CSV
38:   web: ^1.1.1
39:
40: dev_dependencies:
41:   flutter_test:
42:     sdk: flutter
43:
44:   # Règles de qualité de code officielles Flutter
45:   flutter_lints: ^5.0.0
46:
47: flutter:
48:   uses-material-design: true
```

ORGANISATION MODULAIRE

Section 3 sur 16 : cartographie lib/, screens et services

1 lib/screens/	2 lib/services/	3 test/	4 scripts/ & CI
16 écrans d'interface et navigation réactive	16 services de données et DTOs typés	Tests unitaires et de widgets automatisés	check.sh et build.sh pour validation continue

1. **Écrans UI (lib/screens) :** Composent les vues du menu latéral, les formulaires d'édition et les tables de bord.
2. **Services Métier (lib/services) :** Gèrent la sérialisation, les formats monétaires DZD et les appels RPC / Edge Functions.
3. **Suite de Tests (test) :** Valide le parsing des modèles, la robustesse aux données manquantes et les formulaires.
4. **Automatisation (scripts) :** Exécute l'analyse statique Dart, le formatage obligatoire et le build Vercel.

SÉGRÉGATION DES COUCHES

Un écran UI ne doit jamais manipuler directement des requêtes SQL brutes si une entité métier existe.

Les fichiers de services doivent être indépendants du contexte d'affichage (pas de BuildContext dans les parsers).

Les téléchargements de fichiers sont encapsulés sous `financial_report_download.dart` via imports conditionnels.

lib/services/admin_dashboard_service.dart **Lignes 3-35**

```
3: class AdminDashboardOverview {
4:   final int totalUsers;
5:   final int newUsersLastSevenDays;
6:   final int students;
7:   final int teachers;
8:   final int administrators;
9:   final int courses;
10:   final int publishedCourses;
11:   final int draftCourses;
12:   final int lessons;
13:   final int formations;
14:   final int pendingPayments;
15:   final int paidEnrollments;
16:   final List> recentUsers;
17:   final List> recentCourses;
18:
19:   const AdminDashboardOverview({
20:     required this.totalUsers,
21:     required this.newUsersLastSevenDays,
22:     required this.students,
23:     required this.teachers,
24:     required this.administrators,
25:     required this.courses,
26:     required this.publishedCourses,
27:     required this.draftCourses,
28:     required this.lessons,
29:     required this.formations,
30:     required this.pendingPayments,
31:     required this.paidEnrollments,
32:     required this.recentUsers,
33:     required this.recentCourses,
```

AUTHENTIFICATION ET ACCÈS

Section 4 sur 16 : Supabase Auth et contrôle RBAC

1 Saisie Compte	2 Vérification Auth	3 Lecture du Rôle	4 Filtrage Staff
Email, mot de passe, validation regex	signInWithPassword() génère le jeton JWT	Lecture du profil en table public.profiles	teacher ou admin : OK student : signOut()

1. **Validation du formulaire :** Vérifie le format de l'e-mail, masque/démasque le mot de passe et soumet via Entrée.
2. **Session Supabase :** Récupère le jeton JWT d'authentification utilisateur persistant.
3. **Extraction du profil :** Interroge `public.profiles` avec l'UUID utilisateur pour obtenir le rôle et le nom.
4. **Répudiation des étudiants :** Si le rôle n'est ni `teacher` ni `admin`, la session est immédiatement révoquée.

RÈGLE CRITIQUE D'ACCÈS AU STAFF

L'accès à l'application est strictement interdit aux apprenants (rôle `student`).

En cas de tentative de connexion d'un étudiant, l'application appelle `supabase.auth.signOut()` et affiche un message explicite.

Le rôle et le nom validés sont transmis au `DashboardScreen` pour adapter l'interface.

lib/screens/login_screen.dart — **Lignes 30-61**

```
30: final supabase = Supabase.instance.client;
31:
32: final response = await supabase.auth.signInWithPassword(
33:   email: emailController.text.trim(),
34:   password: passwordController.text,
35: );
36:
37: if (response.user == null) {
38:   throw Exception('Login failed.');
39: }
40:
41: final profile = await supabase
42:     .from('profiles')
43:     .select('role, full_name')
44:     .eq('id', response.user!.id)
45:     .single();
46:
47: final role = profile['role'];
48:
49: if (role != 'teacher' && role != 'admin') {
50:   await supabase.auth.signOut();
51:   throw Exception('You are not authorized to access Evolve Staff.');
52: }
53:
54: if (!mounted) return;
55:
56: Navigator.pushReplacement(
57:   context,
58:   MaterialPageRoute(
59:     builder: (_) =>
60:         DashboardScreen(role: role, fullName: profile['full_name'] ?? ''),
```

TABLEAU DE BORD PRINCIPAL

Section 5 sur 16 : hub adaptatif formateur vs administrateur

1 Indicateurs Clés	2 Aperçu Admin	3 Menu Latéral	4 Flux Récent
Cours, étudiants, formations, leçons	Nouveaux comptes, brouillons, paiements en attente	Navigation vers les 10 espaces métier	Derniers cours créés et activités staff

1. **Compteurs pédagogiques :** Affiche en temps réel le total des cours, étudiants inscrits, parcours et leçons.
2. **Blocs réservés aux administrateurs :** Suivi des comptes des 7 derniers jours, paiements et cours à modérer.
3. **Menu contextuel :** Les sections *Users & Roles* et *Revenue* sont masquées aux formateurs simples.
4. **Actualisation transparente :** Bouton de rafraîchissement manuel rechargeant données Supabase et Edge Functions.

ADAPTABILITÉ ET EXPÉRIENCE UTILISATEUR

Les cartes de métriques sont des raccourcis cliquables ouvrant directement les écrans de gestion associés.

La carte *Courses needing review* filtre immédiatement le catalogue sur les cours en statut brouillon.

En cas d'échec réseau, l'interface affiche un état d'erreur explicite sans planter l'application.

lib/screens/dashboard_screen.dart **Lignes 51-78**

```
51: Future _loadDashboardData() async {
52:   setState(() { loading = true; errorMessage = null; });
53:   try {
54:     final overview = widget.role == 'admin'
55:         ? await AdminDashboardOverview.fetch()
56:         : null;
57:     final stats = overview == null
58:         ? await AnalyticsService.fetchStats(widget.role)
59:         : {
60:             'courses': overview.courses,
61:             'students': overview.students,
62:             'formations': overview.formations,
63:             'lessons': overview.lessons,
64:           };
65:     List> recents = [];
66:     if (overview != null) {
67:       recents = overview.recentCourses;
68:     } else {
69:       final res = await supabase
70:           .from('courses')
71:           .select('id, title, domain, created_at')
72:           .order('created_at', ascending: false)
73:           .limit(5);
74:       recents = List>.from(res);
75:     }
76:     if (!mounted) return;
77:     setState(() { courses = stats['courses'] ?? 0; recentActivities = recents; loading = false; });
```

GESTION ET MODÉRATION COURS

Section 6 sur 16 : cycle de vie brouillon, révision et publication

1 Création de Cours	2 Statut Brouillon	3 Modération Admin	4 Mise en Ligne
Titre, description, domaine et prix DZD	published = false invisible aux élèves	Revue pédagogique et approbation admin	published = true visible sur le site

1. **Création de cours :** Le formateur définit les métadonnées, assigne une miniature et fixe le prix catalogue en DZD.
2. **Protection par défaut :** Tout nouveau cours démarre en brouillon (`published: false`) et reste masqué aux étudiants.
3. **Contrôle de publication :** Seul un administrateur peut basculer un cours vers le statut publié via la modération.
4. **Recherche et filtres :** Filtrage multicritères par domaine, statut (tous / brouillons / publiés) et saisie textuelle.

POLITIQUE DE MODÉRATION

Un formateur ne peut pas auto-publier son cours : verrouillé par l'UI ET par le trigger SQL `guard_course_publication` (erreur 42501).

La dépublication d'un cours existant nécessite une confirmation explicite pour éviter d'interrompre les étudiants inscrits.

Les opérations de modération administrative transitent par `AdminCourseModerationService`.

lib/screens/courses_screen.dart **Lignes 40-72**

```
40: Future loadCourses() async {
41:   try {
42:     final user = supabase.auth.currentUser;
43:     if (user == null) throw Exception('User is not logged in.');
44:
45:     final profile = await supabase
46:         .from('profiles')
47:         .select('role')
48:         .eq('id', user.id)
49:         .single();
50:     isAdmin = profile['role'] == 'admin';
51:
52:     List> res = [];
53:     if (isAdmin) {
54:       res = await AdminCoursesService.fetchCourses();
55:     } else {
56:       final data = await supabase
57:           .from('courses')
58:           .select('*, lessons(count)')
59:           .eq('teacher_id', user.id)
60:           .order('created_at', ascending: false);
61:       res = List>.from(data);
62:     }
63:     if (!mounted) return;
64:     setState(() { allCourses = res; loading = false; });
65:     filterCourses();
66:   } catch (e) {
67:     setState(() { error = e.toString(); loading = false; });
68:   }
69: }
```

LEÇONS ET RESSOURCES DE COURS

Section 7 sur 16 : structuration des chapitres et upload Storage

1 Ordre et Titre	2 Contenu Vidéo	3 Pièces Jointes	4 Métadonnées
order_index strict pour le parcours séquentiel	video_url et durée estimée en minutes	Upload Supabase Storage (PDF, code, archives)	Table course_files pour traçabilité des fichiers

1. **Organisation séquentielle :** Chaque leçon possède un `order_index` déterminant la progression sur le portail étudiant.
2. **Ressources vidéo :** Lien d'intégration vidéo (YouTube ou hébergement dédié) associé à une durée calibrée.
3. **Stockage de documents :** `course_files_screen.dart` utilise `file_picker` pour téléverser vers le bucket Storage.
4. **Référencement en base :** Chaque fichier uploadé génère une ligne dans `course_files` liant URL, nom et taille.

INTÉGRITÉ DU CONTENU PÉDAGOGIQUE

L'accès aux leçons protégées sur le site web nécessite une inscription payée (`payment_status = 'paid'`).

La suppression d'une leçon cascade les enregistrements de progression (`lesson_progress`) associés.

Les fichiers de cours sont servis via des URLs publiques ou signées selon la visibilité du bucket.

lib/screens/course_files_screen.dart — **Lignes 35-65**

```
35: Future _uploadFile() async {
36:   final result = await FilePicker.platform.pickFiles(withData: true);
37:   if (result == null || result.files.isEmpty) return;
38:
39:   final file = result.files.first;
40:   if (file.bytes == null) return;
41:
42:   setState(() => _uploading = true);
43:   try {
44:     final path = '${widget.courseId}/${DateTime.now().millisecondsSinceEpoch}_${file.name}';
45:     await supabase.storage.from('course_materials').uploadBinary(path, file.bytes!);
46:     final fileUrl = supabase.storage.from('course_materials').getPublicUrl(path);
47:
48:     await supabase.from('course_files').insert({
49:       'course_id': widget.courseId,
50:       'name': file.name,
51:       'file_url': fileUrl,
52:       'size_bytes': file.size,
53:       'uploaded_by': supabase.auth.currentUser!.id,
54:     });
55:
56:     if (!mounted) return;
57:     _loadFiles();
58:   } catch (e) {
59:     ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text('Erreur: $e')));
60:   } finally {
61:     if (mounted) setState(() => _uploading = false);
62:   }
63: }
```

FORMATIONS ET ATELIERS

Section 8 sur 16 : séries de cours et masterclasses pratiques

1 Parcours Séries	2 Ordonnancement	3 Ateliers Pratiques	4 Certifications
course_series : regroupe plusieurs cours liés	Progression séquentielle obligatoire de la série	Sessions immersives, replay et projets	Diplôme délivré dès complétion intégrale

1. **Entité Formations (course_series) :** Regroupe plusieurs cours dans un cursus thématique cohérent.
2. **Verrouillage séquentiel :** Un étudiant ne peut aborder le cours N+1 d'une série sans avoir validé à 100 % le cours N.
3. **Workshops / Ateliers :** Modules intensifs gérés via `workshops_screen.dart` avec suivi de projet mentoré.
4. **Émission des attestations :** La complétion globale d'une formation déclenche la génération d'un certificat officiel.

RÈGLES D'ORGANISATION DES FORMATIONS

Les associations cours-série sont sécurisées en base par les politiques RLS de `course_series_rls.sql`.

Un cours peut être proposé individuellement ou au sein d'un parcours de formation complet.

Le statut d'inscription aux ateliers est tracé distinctement dans `workshop_enrollments`.

lib/services/formation_service.dart **Lignes 1-32**

```
1: import 'package:supabase_flutter/supabase_flutter.dart';
2:
3: class Formation {
4:   final String id;
5:   final String title;
6:   final String? description;
7:   final String? domain;
8:   final List courseIds;
9:
10:   Formation({
11:     required this.id,
12:     required this.title,
13:     this.description,
14:     this.domain,
15:     this.courseIds = const [],
16:   });
17:
18:   factory Formation.fromMap(Map map) {
19:     return Formation(
20:       id: map['id'],
21:       title: map['title'] ?? '',
22:       description: map['description'],
23:       domain: map['domain'],
24:       courseIds: (map['courses'] as List?)
25:           ?.map((c) => c['id'].toString())
26:           .toList() ?? [],
27:     );
28:   }
29: }
```

DEVOIRS ET NOTATION

Section 9 sur 16 : création, soumission et correction formative

1 Création Devoir	2 Dépôts Étudiants	3 Évaluation Staff	4 Statut de Revue
Titre, consignes et date limite d'échéance	assignment_submissions : liens et fichiers rendus	Attribution d'une note et commentaire feedback	pending -> reviewed visible par l'étudiant

1. **Configuration des assignations :** Le formateur crée les devoirs rattachés à un cours précis.
2. **Réception des travaux :** Vue centralisée de tous les devoirs soumis par les apprenants dans `assignments_screen.dart`.
3. **Grille d'évaluation :** Le formateur saisit la note chiffrée et un feedback qualitatif constructif.
4. **Notification de l'élève :** Le statut de la soumission bascule à `reviewed` et apparaît sur l'espace élève.

PRINCIPES D'ÉVALUATION

Un formateur ne peut corriger que les devoirs associés aux cours dont il est l'instructeur référent.

Les administrateurs ont une visibilité complète sur la file des devoirs en attente de correction.

Le service `AssignmentService` fournit un mode témoin de repli en cas d'indisponibilité temporaire de la base.

lib/services/assignment_service.dart **Lignes 25-50**

```
25: class AssignmentSubmission {
26:   final String id;
27:   final String assignmentId;
28:   final String studentId;
29:   final String? studentName;
30:   final String contentUrl;
31:   final String status; // 'pending' | 'reviewed'
32:   final double? grade;
33:   final String? feedback;
34:   final DateTime submittedAt;
35:
36:   AssignmentSubmission({
37:     required this.id,
38:     required this.assignmentId,
39:     required this.studentId,
40:     this.studentName,
41:     required this.contentUrl,
42:     required this.status,
43:     this.grade,
44:     this.feedback,
45:     required this.submittedAt,
46:   });
47:
48:   static Future gradeSubmission(String id, double grade, String feedback) async {
49:     await Supabase.instance.client.from('assignment_submissions').update({
50:       'grade': grade, 'feedback': feedback, 'status': 'reviewed',
51:     }).eq('id', id);
52:   }
53: }
```

PROGRESSION DES ÉTUDIANTS

Section 10 sur 16 : complétion temps réel et attestation

1 Table de Suivi	2 Taux de Réussite	3 Filtres et Recherche	4 Délivrance Diplôme
lesson_progress : trace chaque leçon validée	Calcul du % de complétion par cours et par élève	Par cours, nom d'élève ou état d'avancement	Validation 100 % requise pour le certificat

1. **Granularité par leçon :** La table `lesson_progress` enregistre la dernière position et le statut `completed`.
2. **Agrégation d'avancement :** `StudentProgressService` compare les leçons achevées au volume total du cours.
3. **Vue pédagogique Staff :** Permet d'identifier les élèves en difficulté ou bloqués sur un chapitre spécifique.
4. **Critère d'attestation :** Le certificat de formation n'est accessible que lorsque 100 % des leçons obligatoires sont cochées.

MOTEUR D'AVANCEMENT PÉDAGOGIQUE

Les cours sans leçons publiées retournent un taux de 0 % pour éviter toute division par zéro.

Les données de progression sont en lecture seule pour les formateurs et mises à jour par l'activité de l'étudiant.

La recherche par nom d'étudiant supporte la pagination et les caractères spéciaux.

lib/services/student_progress_service.dart **Lignes 20-45**

```
20: class StudentProgressSummary {
21:   final String studentId;
22:   final String studentName;
23:   final String courseId;
24:   final String courseTitle;
25:   final int completedLessons;
26:   final int totalLessons;
27:   final double progressPercentage;
28:
29:   StudentProgressSummary({
30:     required this.studentId,
31:     required this.studentName,
32:     required this.courseId,
33:     required this.courseTitle,
34:     required this.completedLessons,
35:     required this.totalLessons,
36:     required this.progressPercentage,
37:   });
38:
39:   factory StudentProgressSummary.calculate({
40:     required Map student,
41:     required Map course,
42:     required int completed,
43:     required int total,
44:   }) {
45:     final percentage = total > 0 ? (completed / total) * 100.0 : 0.0;
46:     return StudentProgressSummary(
47:       studentId: student['id'],
48:       studentName: student['full_name'] ?? 'Inconnu',
49:       courseId: course['id'],
50:       courseTitle: course['title'] ?? '',
```

MESSAGERIE INSTANTANÉE TEMPS RÉEL

Section 11 sur 16 : abonnements WebSockets et isolation des canaux

1 Canal WebSocket	2 Tri et Non Lus	3 Sécurité RLS	4 Réponse Directe
Supabase Realtime sur table direct_messages	Filtre sur messages non lus et tri chronologique	Seuls émetteur et récepteur accèdent au contenu	Envoi instantané répercuté sur Evolve School

1. **Abonnement temps réel :** Le contrôleur souscrit à `PostgresChangeEvent.insert` pour actualiser la boîte de réception.
2. **Filtrage efficace :** Permet d'isoler en un clic les discussions avec des messages en attente de réponse.
3. **Confidentialité des échanges :** Protégée par les règles RLS de `20260930203100_student_teacher_messaging.sql`.
4. **Synchronisation avec le site :** La réponse envoyée depuis le Staff apparaît sur le portail étudiant en temps réel.

RÈGLES D'ISOLATION DES COMMUNICATIONS

Un étudiant ne peut initier de message qu'avec un formateur d'un cours auquel il est activement inscrit.

Le canal WebSocket est désinscrit proprement dans la méthode `dispose()` pour éviter les fuites mémoire.

Les conversations sont initialisées dès le premier message transmis par l'élève depuis Evolve School.

lib/screens/messages_screen.dart **Lignes 20-43**

```
20: @override
21: void initState() {
22:   super.initState();
23:   _loadConversations();
24:   _messagesChannel = _supabase
25:       .channel('staff_direct_messages_inbox')
26:       .onPostgresChanges(
27:         event: PostgresChangeEvent.insert,
28:         schema: 'public',
29:         table: 'direct_messages',
30:         callback: (_) {
31:           if (mounted) _loadConversations(showLoading: false);
32:         },
33:       )
34:       .subscribe();
35: }
36:
37: @override
38: void dispose() {
39:   if (_messagesChannel != null) {
40:     _supabase.removeChannel(_messagesChannel!);
41:   }
42:   super.dispose();
43: }
```

GESTION UTILISATEURS ET RBAC

Section 12 sur 16 : administration des profils et invitations

1 Catalogue Profils Edge Function admin-users avec recherche complète	**2 Changement Rôle** admin_update_profile_role via procédure stockée	**3 Invitation Admin** Edge Function invite-admin avec e-mail d'activation	**4 Garde Verrouillage** Dernier admin protégé et anti-auto-rétrogradation

1. **Listing sécurisé :** L'Edge Function `admin-users` contourne le RLS utilisateur tout en validant le JWT admin.
2. **Modification de privilèges :** Seuls les rôles `student`, `teacher` et `admin` sont supportés.
3. **Invitation par courriel :** Envoie une invitation Supabase Auth créant un compte admin dès acceptation.
4. **Protection de stabilité :** La base interdit formellement de retirer les droits du dernier administrateur.

RÈGLE CRITIQUE D'ADMINISTRATION DES RÔLES

Un administrateur ne peut jamais rétrograder son propre compte (protection contre l'auto-exclusion).

La mise à jour de rôle applique un verrou de table PostgreSQL pour éliminer toute condition de course critique.

L'écran `UsersManagementScreen` est strictement masqué aux utilisateurs n'ayant pas le rôle `admin`.

supabase/migrations/20261003_secure_profile_role_management.sql **Lignes 50-70**

```
50: IF p_new_role IS NULL OR p_new_role NOT IN ('student', 'teacher', 'admin') THEN
51:   RAISE EXCEPTION 'Unsupported profile role' USING ERRCODE = '22023';
52: END IF;
53:
54: IF NOT EXISTS (
55:   SELECT 1 FROM public.profiles AS actor
56:   WHERE actor.id = p_actor_id AND actor.role = 'admin'
57: ) THEN
58:   RAISE EXCEPTION 'Administrator access required' USING ERRCODE = '42501';
59: END IF;
60:
61: IF p_actor_id = p_target_user_id AND p_new_role <> 'admin' THEN
62:   RAISE EXCEPTION 'Administrators cannot remove their own admin access'
63:     USING ERRCODE = '42501';
64: END IF;
65:
66: LOCK TABLE public.profiles IN SHARE ROW EXCLUSIVE MODE;
```

MODULE FINANCIER ET REVENUS

Section 13 sur 16 : réconciliation, dépenses et remboursements

1 Revenus Réels DZD	2 Anciens Paiements	3 Dépenses Exploitation	4 Remboursements
Montants conservés au moment du paiement	Signalés à part pour éviter toute extrapolation fausse	Saisie manuelle des frais serveurs et prestations	Obligatoirement liés à un enrollment_id payé

1. **Réconciliation des recettes :** Calculé sur la base exacte des montants perçus via Chargily Pay en Dinars (DZD).
2. **Intégrité historique :** Les anciens paiements sans montant conservé ne sont pas estimés au prix catalogue actuel.
3. **Grand livre des écritures :** Table `admin_financial_entries` pour enregistrer les charges et ajustements.
4. **Garde-fou de remboursement :** La base refuse tout remboursement supérieur au montant initialement versé.

RÈGLES COMPTABLES ET RIGUEUR MONÉTAIRE

Les montants monétaires sont stockés sous forme d'unités mineures (centimes) pour écarter toute erreur d'arrondi.

Les alertes financières signalent : baisse anormale des encaissements, pic d'inscriptions en attente et désistements.

Les écritures ne constituent pas un bilan comptable fiscal (les frais bancaires Chargily ne sont pas déduits automatiquement).

supabase/migrations/20261004_financial_reporting.sql **Lignes 8-22**

```
8: create table if not exists public.admin_financial_entries (
9:   id uuid primary key default gen_random_uuid(),
10:   entry_type text not null check (entry_type in ('expense', 'refund')),
11:   amount_minor_units bigint not null check (amount_minor_units > 0),
12:   category text not null check (length(trim(category)) between 1 and 80),
13:   note text not null check (length(trim(note)) between 1 and 500),
14:   occurred_at timestamptz not null,
15:   enrollment_id uuid references public.enrollments(id) on delete restrict,
16:   recorded_by uuid not null references public.profiles(id) on delete restrict,
17:   created_at timestamptz not null default now(),
18:   constraint admin_financial_entry_enrollment_type check (
19:     (entry_type = 'expense' and enrollment_id is null)
20:     or (entry_type = 'refund' and enrollment_id is not null)
21:   )
22: );
```

EXPORTATION RAPPORTS FINANCIERS

Section 13.2 sur 16 : téléchargement CSV multiplateforme

1 Compilation Dart	2 Export Web	3 Export Desktop	4 Compatibilité Excel
Imports conditionnels : _web.dart vs _stub.dart	Blob UTF-8 + BOM et balise a HTML temporaire	Écriture sur disque local sans dépendance HTML	Préfixe \uFEFF pour encodage UTF-8 sous Windows

1. **Architecture conditionnelle :** Le fichier `financial_report_download.dart` sélectionne la cible selon la plateforme.
2. **Implémentation Web :** Génère un Object URL via `web.Blob`, clique sur l'ancre et révoque l'URL après 1 seconde.
3. **Implémentation Windows / Tests :** Utilise un stub sécurisé pour ne pas provoquer d'erreur lors des tests CLI.
4. **Formatage du CSV :** Intègre le Byte Order Mark (BOM) UTF-8 pour un affichage parfait dans Microsoft Excel.

RÈGLE D'ISOLATION DES PACKAGES WEB

Ne jamais importer `dart:html` ou `package:web` directement dans un écran commun.

Tout accès au Document Object Model du navigateur doit être encapsulé derrière une façade Dart conditionnelle.

Cette règle garantit que `flutter test` et le build Windows natif s'exécutent sans erreur de compilation.

lib/services/financial_report_download_web.dart **Lignes 1-26**

```
1: import 'dart:async';
2: import 'dart:js_interop';
3:
4: import 'package:web/web.dart' as web;
5:
6: bool downloadFinancialReport(String filename, String csv) {
7:   final blob = web.Blob(
8:     ['\uFEFF$csv'.toJS].toJS,
9:     web.BlobPropertyBag(type: 'text/csv;charset=utf-8'),
10:   );
11:   final url = web.URL.createObjectURL(blob);
12:   final anchor = web.HTMLAnchorElement()
13:     ..href = url
14:     ..download = filename
15:     ..style.display = 'none';
16:   final body = web.document.body;
17:   if (body == null) {
18:     web.URL.revokeObjectURL(url);
19:     return false;
20:   }
21:   body.append(anchor);
22:   anchor.click();
23:   anchor.remove();
24:   Timer(const Duration(seconds: 1), () => web.URL.revokeObjectURL(url));
25:   return true;
26: }
```

EDGE FUNCTIONS DÉDIÉES

Section 14 sur 16 : architecture des endpoints Deno/TypeScript

1 admin-dashboard Agrégation globale des statistiques et cours	**2 admin-users** Listing sécurisé des utilisateurs et rôles	**3 admin-financials** Consolidation du grand livre et écritures DZD	**4 admin-moderation** Approbation et publication des cours staff

1. **Agrégation d'accueil (admin-dashboard) :** Calcule les inscriptions, revenus et volumes sans exposer les tables privées.
2. **Gestion des utilisateurs (admin-users) :** Pagination et recherche plein texte sans risque de fuite de données personnelles.
3. **Grand livre financier (admin-financials) :** Isole la lecture comptable et vérifie les montants de remboursement.
4. **Modération (admin-courses & moderation) :** Permet aux administrateurs de relire l'intégralité des brouillons.

SÉCURISATION DES MICROSERVICES DENO

Chaque Edge Function vérifie le token Bearer JWT de l'utilisateur appelant dès réception de la requête HTTP.

Si le rôle dans `public.profiles` n'est pas `admin`, la fonction répond immédiatement par un code HTTP 403 Forbidden.

Les variables sensibles (`SUPABASE_SERVICE_ROLE_KEY`) ne sont accessibles qu'à l'environnement d'exécution du serveur Deno.

lib/services/admin_analytics_service.dart **Lignes 1-32**

```
1: import 'package:supabase_flutter/supabase_flutter.dart';
2:
3: class AdminAnalyticsService {
4:   static Future> fetchAnalytics() async {
5:     final supabase = Supabase.instance.client;
6:     final session = supabase.auth.currentSession;
7:     if (session == null) throw Exception('Session non authentifiée');
8:
9:     final response = await supabase.functions.invoke(
10:       'admin-analytics',
11:       headers: {
12:         'Authorization': 'Bearer ${session.accessToken}',
13:       },
14:     );
15:
16:     if (response.status != 200) {
17:       throw Exception('Erreur analytique: ${response.status}');
18:     }
19:
20:     final data = response.data;
21:     if (data is! Map) {
22:       throw FormatException('Réponse analytique invalide');
23:     }
24:     return data;
25:   }
26: }
```

SÉCURITÉ RLS ET MODÈLE MENACE

Section 15 sur 16 : triggers de protection et politiques de restriction

❶ Zéro Rôle Modifiable	❷ RLS sur les Cours	❸ RLS Inscriptions	❹ RLS Messagerie
Trigger guard_profile_role bloque toute élévation	Formateur : ses cours Admin : lecture intégrale	L'étudiant ne peut pas s'auto-passer en 'paid'	Restreint aux participants de la discussion active

❶ **Blocage de l'élévation de privilèges :** Un compte authentifié ne peut pas s'attribuer le rôle `admin` via un UPDATE.

❷ **Isolation des cours :** La politique SELECT restreint l'édition des brouillons au formateur propriétaire.

❸ **Inviolabilité des paiements :** Seul le webhook serveur ou une fonction admin peut inscrire `payment_status = 'paid'`.

❹ **Cloisonnement des messages :** La lecture et l'écriture exigent que l'UUID appelant corresponde à l'émetteur ou au destinataire.

MODÈLE DE MENACE ET DÉFENSE EN PROFONDEUR

Le client Flutter est considéré comme non fiable : aucune validation de sécurité critique ne repose sur du code Dart.

La base de données PostgreSQL applique systématiquement le principe de moindre privilège via Row-Level Security.

Toute violation de règle RLS se traduit par un retour vide silencieux ou une exception SQL immédiate (erreur 42501).

supabase/migrations/20261003_secure_profile_role_management.sql **Lignes 1-29**

```
1: CREATE OR REPLACE FUNCTION public.guard_profile_role_assignments()
2: RETURNS trigger
3: LANGUAGE plpgsql
4: AS $$
5: BEGIN
6:   IF auth.role() = 'authenticated' THEN
7:     IF TG_OP = 'INSERT' AND NEW.role IS DISTINCT FROM 'student' THEN
8:       RAISE EXCEPTION 'New accounts cannot assign their own privileged role'
9:         USING ERRCODE = '42501';
10:     END IF;
11:
12:     IF TG_OP = 'UPDATE' AND NEW.role IS DISTINCT FROM OLD.role THEN
13:       RAISE EXCEPTION 'Profile roles can only be changed by an administrator'
14:         USING ERRCODE = '42501';
15:     END IF;
16:   END IF;
17:
18:   RETURN NEW;
19: END;
20: $$;
21:
22: DROP TRIGGER IF EXISTS guard_profile_role_assignments ON public.profiles;
23:
24: CREATE TRIGGER guard_profile_role_assignments
25:   BEFORE INSERT OR UPDATE OF role
26:   ON public.profiles
27:   FOR EACH ROW
28:   EXECUTE FUNCTION public.guard_profile_role_assignments();
```

ASSURANCE QUALITÉ ET TESTS

Section 16 sur 16 : tests unitaires, de widget et scripts CI

1 Vérification Lockfile	2 Formatage Dart	3 Analyse Statique	4 Tests Automatisés
pub get --enforce-lockfile reproductibilité stricte	dart format officiel sans écart toléré	flutter analyze zéro warning fatal	flutter test : modèles, revenus et widgets

1. **Lockfile inviolable :** Empêche toute dérive silencieuse de version de package lors des compilations distantes.
2. **Formatage standard :** Exécution de `dart format` sur `lib/` et `test/` imposée avant tout commit.
3. **Linter strict :** `flutter_lints 5.0.0` activé avec interdiction de variables inutilisées et de typages lâches.
4. **Tests non-régression :** Couvrent le calcul de complétion des devoirs, la réconciliation DZD et le login.

DISPOSITIFS D'ASSURANCE QUALITÉ CI

Le pipeline GitHub Actions échoue immédiatement si une seule vérification du script `check.sh` renvoie un statut non nul.

Il est strictement interdit de bypasser les avertissements du compilateur Dart avec des commentaires d'ignorance sans justification.

Avant d'ouvrir une Pull Request, le développeur doit valider localement l'exécution de `bash scripts/check.sh`.

scripts/check.sh **Lignes 1-21**

```
1: #!/usr/bin/env bash
2: set -euo pipefail
3:
4: cd "$(dirname "${BASH_SOURCE[0]}")/.."
5:
6: echo "Resolving Flutter dependencies from pubspec.lock..."
7: flutter pub get --enforce-lockfile
8:
9: echo "Checking Dart formatting..."
10: dart format --output=none --set-exit-if-changed lib test
11:
12: echo "Analyzing Flutter application..."
17: flutter analyze --no-pub --no-fatal-infos
18:
19: echo "Running Flutter tests..."
20: flutter test --no-pub
21:
```

BUILDS ET DÉPLOIEMENT PRODUCTION

Section 16.2 sur 16 : release Web Vercel et runner Windows

❶ Hébergement Vercel	❷ Routing SPA Web	❸ En-têtes Cache	❹ Exécutable Win32
scripts/build.sh installe Flutter SDK 3.35.7	vercel.json : réécriture de toutes routes vers /	no-cache sur index.html et service worker	flutter build windows release native desktop

❶ **Environnement Vercel :** Le script `build.sh` télécharge automatiquement le tag officiel Flutter et compile en Web Release.

❷ **Redirections SPA :** `vercel.json` réécrit toutes les requêtes d'URL vers `index.html` pour préserver le routeur.

❸ **Protection contre le cache obsolète :** `no-store` forcé sur `version.json` et `flutter_service_worker.js`.

❹ **Binaire natif Windows :** Produit un exécutable portable Win32 haute performance dans `build/windows/x64/`.

DIRECTIVES DE DÉPLOIEMENT

Ne jamais intégrer de variables d'environnement secrètes dans les variables publiques de build Vercel.

Le déploiement web produit une arborescence statique servie depuis le dossier `build/web`.

En cas de régression en production, le panneau Vercel permet d'effectuer un rollback instantané en 1 clic.

vercel.json **Lignes 1-22**

```
1: {
2:   "buildCommand": "bash scripts/build.sh",
3:   "outputDirectory": "build/web",
4:   "rewrites": [
5:     {
6:       "source": "/(.*)",
7:       "destination": "/index.html"
8:     }
9:   ],
10:   "headers": [
11:     {
12:       "source": "/(index.html|flutter_service_worker.js|version.json)",
13:       "headers": [
14:         {
15:           "key": "Cache-Control",
16:           "value": "no-cache, no-store, must-revalidate"
17:         }
18:       ]
19:     }
20:   ]
21: }
```

DÉBOGAGE ET EXPLOITATION

Section 16.3 sur 16 : diagnostic opérationnel et FAQ des erreurs

1 Accès non autorisé Profil sans rôle ou rôle student configuré	**2 Chargement infini** Edge Function manquante ou session expirée	**3 Échec Export CSV** Blocage pop-up du navigateur sur le web	**4 Brouillons bloqués** Nécessite la validation d'un compte admin

1. **Erreur "You are not authorized" :** Vérifier la table `public.profiles` et attribuer `teacher` ou `admin`.
2. **Blocage écran Dashboard :** Consulter les logs Supabase Edge Functions ; vérifier que les secrets JWT sont valides.
3. **Téléchargement inactif :** Vérifier les autorisations de téléchargement automatique sur le domaine de l'application.
4. **Cours invisible aux étudiants :** Un formateur ne peut pas auto-publier son cours ; l'admin doit l'approuver.

PROTOCOLE DE DÉPANNAGE EN PRODUCTION

En cas de signalement d'incident, vérifier en priorité la connectivité Supabase et la validité du token utilisateur.

Ne jamais communiquer de token d'authentification ou d'identifiant sensible dans un ticket de support.

Utiliser le bouton **Refresh** de chaque écran pour forcer la synchronisation avec la base PostgreSQL.

README.md **Lignes 111-128**

````
111: ### Si un écran ne charge pas
112:
113: 1. Vérifiez votre connexion Internet et votre session.
114: 2. Utilisez **Refresh** ou **Try again** sur l’écran.
115: 3. Vérifiez que le rôle du compte et les données existent dans Supabase.
116: 4. Si l’erreur persiste, transmettez son texte à l’administrateur ; ne collez
117:    jamais de clé secrète ou de jeton d’authentification dans un ticket.
118:
119: ## Développement et vérifications
120:
121: Avant de proposer des changements :
122:
123: ```bash
124: dart format lib test
125: flutter analyze --no-pub --no-fatal-infos
126: flutter test --no-pub
127: ```
````

AUDIT DU CODE ET ÉCARTS

Section 16.4 sur 16 : dette technique et préconisations de refactorisation

1 Fichiers denses	2 Gestion d'état	3 Copies de sauvegarde	4 Conformité Globale
dashboard_screen.dart (1 205 lignes à scinder)	StatefulWidgets locaux ; évolutif vers Provider/Bloc	workshops_screen.dart.bak à purger du dépôt	Zéro fuite de secret, sound null-safety 100 %

1. **Volume de code par écran :** `dashboard_screen.dart` (1 205 l.) et `assignments_screen.dart` (881 l.) doivent être découpés en sous-widgets.
2. **Gestion d'état global :** La gestion par `setState` locale est robuste mais gagnerait à adopter un pattern Riverpod ou Bloc.
3. **Nettoyage des fichiers temporaires :** Supprimer le fichier `workshops_screen.dart.bak` non suivi.
4. **Sécurité et intégrité :** Audit de sécurité 100 % conforme : aucune clé `service_role` ni secret bancaire dans le code.

RECOMMANDATIONS D'ÉVOLUTION TECHNIQUE

Découper les écrans de plus de 500 lignes en composants spécialisés réutilisables dans un dossier `lib/widgets/`.

Ajouter une suite de tests d'intégration simulant le parcours complet de modération et de notation des devoirs.

Maintenir la compatibilité stricte avec les futures versions majeures de Flutter et du package `supabase_flutter`.

analysis_options.yaml **Lignes 1-28**

```
1: include: package:flutter_lints/flutter.yaml
2:
3: linter:
4:   rules:
5:     # Encourage de bonnes pratiques de lisibilité et de sécurité
6:     avoid_print: true
7:     prefer_const_constructors: true
8:     prefer_const_literals_to_create_immutables: true
9:     unawaited_futures: true
10:     cancel_subscriptions: true
11:
12: analyzer:
13:   errors:
14:     missing_required_param: error
15:     missing_return: error
16:     todo: ignore
17:   exclude:
18:     - "build/**"
19:     - "**/*.g.dart"
```

INDEX EXHAUSTIF DES FICHIERS (1)

Annexe A · Section 16 sur 16 : cartographie lib/screens et lib/services

Chemin du fichier	Rôle & Responsabilité
lib/main.dart	Point d'entrée, initialisation Supabase et runApp()
lib/screens/login_screen.dart	Authentification e-mail / mot de passe et contrôle de rôle RBAC
lib/screens/dashboard_screen.dart	Tableau de bord adaptatif (compteurs formateur vs admin)
lib/screens/courses_screen.dart	Catalogue de cours, filtres de modération et actions admin
lib/screens/create_course_screen.dart	Formulaire de création d'un cours (titre, prix DZD, domaine)
lib/screens/edit_course_screen.dart	Modification des métadonnées, prix et statut d'un cours existant
lib/screens/course_details_screen.dart	Fiche détaillée du cours, liste des leçons et accès aux pièces jointes
lib/screens/course_files_screen.dart	Upload et gestion des fichiers de cours sur Supabase Storage
lib/screens/formations_screen.dart	Gestion des parcours de formation (séries de cours structurés)
lib/screens/lessons_screen.dart	Listing global des leçons, ordonnancement et ajout
lib/screens/lesson_details_screen.dart	Détail d'une leçon individuelle, durée et vidéo
lib/screens/assignments_screen.dart	Création de devoirs, révision des soumissions et notation
lib/screens/student_progress_screen.dart	Suivi de la progression des apprenants et taux de complétion
lib/screens/messages_screen.dart	Messagerie instantanée bidirectionnelle en direct avec les élèves
lib/screens/workshops_screen.dart	Gestion des ateliers pratiques immersifs et des masterclasses
lib/screens/workshop_content_screen.dart	Gestion du contenu multimédia et livrables de workshops
lib/screens/users_management_screen.dart	Administration des comptes, modification de rôles et invitations
lib/screens/revenue_screen.dart	Grand livre des revenus en DZD, dépenses et remboursements
lib/screens/admin_analytics_screen.dart	Statistiques avancées, graphiques de tendance et cohortes
lib/services/admin_dashboard_service.dart	Consommation et validation des métriques du dashboard admin
lib/services/admin_analytics_service.dart	Extraction des séries chronologiques via Edge Function
lib/services/admin_courses_service.dart	Chargement de l'ensemble des cours pour examen administratif
lib/services/admin_course_moderation_service.dart	Validation, publication et retrait de cours par les admins

INDEX EXHAUSTIF DES FICHIERS (2)

Chemin du fichier	Rôle & Responsabilité
lib/services/admin_users_service.dart	Appel de l'Edge Function admin-users pour gestion des profils
lib/services/admin_financial_service.dart	Gestion du grand livre, saisie de dépenses et remboursements
lib/services/financial_report_download.dart	Interface conditionnelle de téléchargement de rapport CSV
lib/services/financial_report_download_web.dart	Implémentation Web via Blob et W3C JS Interop (package:web)
lib/services/financial_report_download_stub.dart	Implémentation Desktop/Test sans dépendance au navigateur
lib/services/assignment_service.dart	Modèles et opérations CRUD devoirs, soumissions et notation
lib/services/formation_service.dart	Modélisation et requêtes des parcours (course_series)
lib/services/revenue_service.dart	Formatage des montants monétaires DZD et réconciliation
lib/services/student_progress_service.dart	Calculs de taux de complétion par étudiant et par cours
lib/services/workshop_service.dart	Gestion des masterclasses et inscriptions aux workshops
lib/services/workshop_content_service.dart	Gestion du contenu interactif des ateliers
lib/services/analytics_service.dart	Comptage direct des cours, leçons et formations
supabase/migrations/20260920_production_resilience_indexes.sql	Index d'optimisation de performance pour la production
supabase/migrations/20260924_orders_and_rls_hardening.sql	Renforcement des politiques RLS sur les commandes
supabase/migrations/20260930203100_student_teacher_messaging.sql	Tables et politiques RLS de la messagerie instantanée
supabase/migrations/20261003_secure_profile_role_management.sql	Trigger anti-élévation et procédure admin_update_profile_role
supabase/migrations/20261004_financial_reporting.sql	Table admin_financial_entries et écritures comptables
supabase/migrations/20261005_course_series_rls.sql	Sécurisation RLS des parcours et séries de cours
supabase/migrations/20261012_guard_course_publication.sql	Trigger SQL interdisant la publication de cours sans rôle admin
supabase/migrations/20261012_chargily_webhook_audit_log.sql	Table d'audit log et traçabilité des transactions Chargily Pay
scripts/check.sh	Script CI de contrôle : lockfile, format, analyse et tests
scripts/build.sh	Script de build Vercel : installation du SDK Flutter et build web
test/admin_dashboard_service_test.dart	Tests unitaires de robustesse du parser de tableau de bord
test/revenue_service_test.dart	Tests du formatage des devises DZD et des règles de grand livre
test/widget_test.dart	Test du formulaire de login et validation des mots de passe